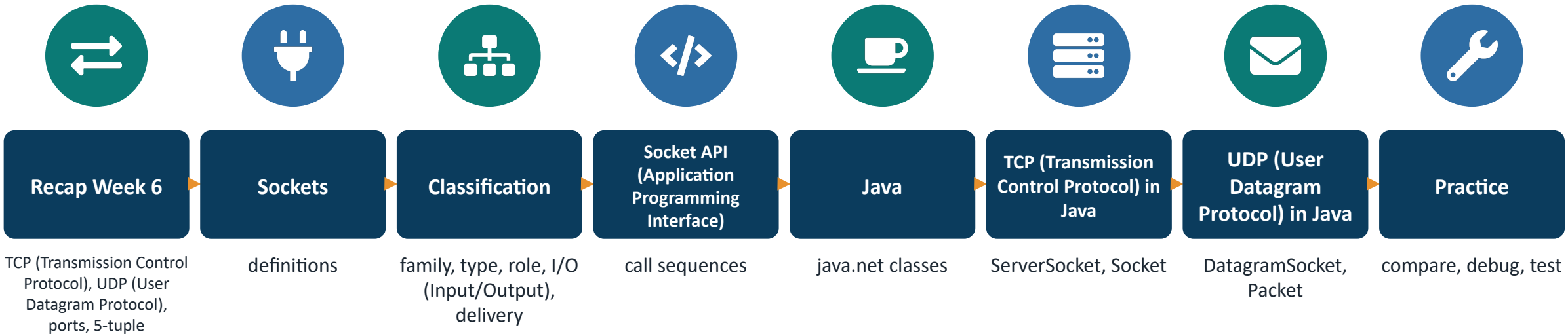
Socket Communication

Week 7 — Schemes for learning: definitions, classification, the socket API (Application Programming Interface), Java TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) client-server programming

Week 7 at a glance: from the transport layer to working programs



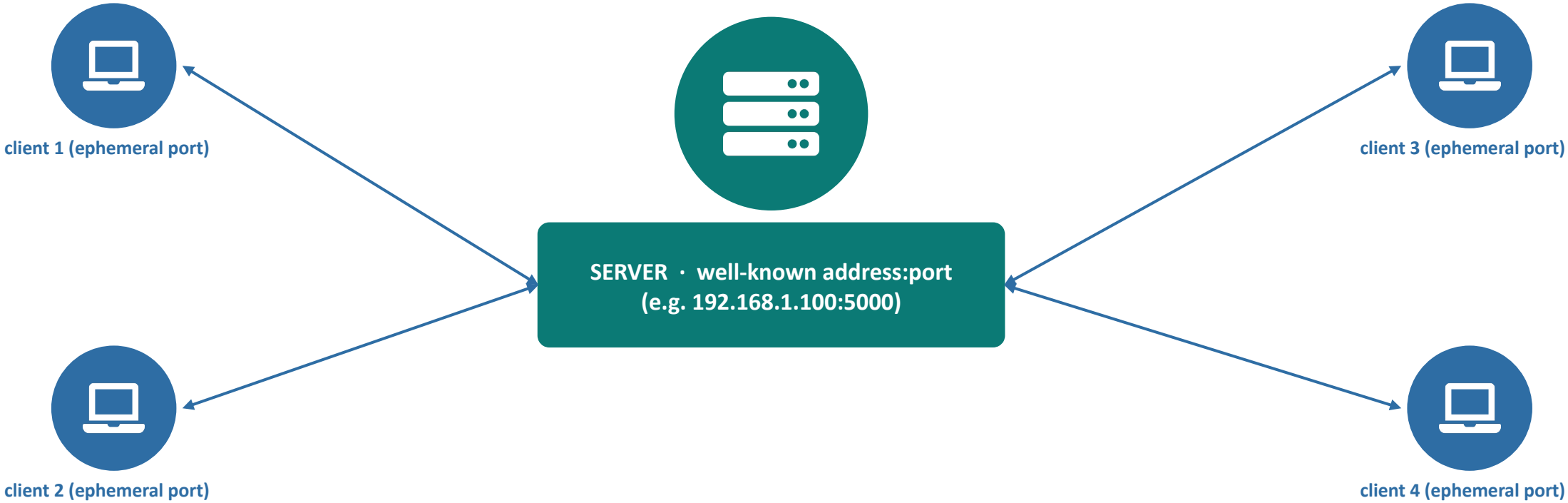
Key idea of the week: applications never build TCP (Transmission Control Protocol) segments or UDP (User Datagram Protocol) datagrams themselves. They ask the operating system for a SOCKET and read/write data through it; the kernel runs TCP (Transmission Control Protocol)/UDP (User Datagram Protocol)/IP (Internet Protocol) underneath. Java wraps the socket API (Application Programming Interface) in a few classes of the java.net package.

1

From transport services to applications

Client-server model and the place of the socket

The client-server model



Server

- passive: waits for requests
- always on, fixed port
- serves many clients

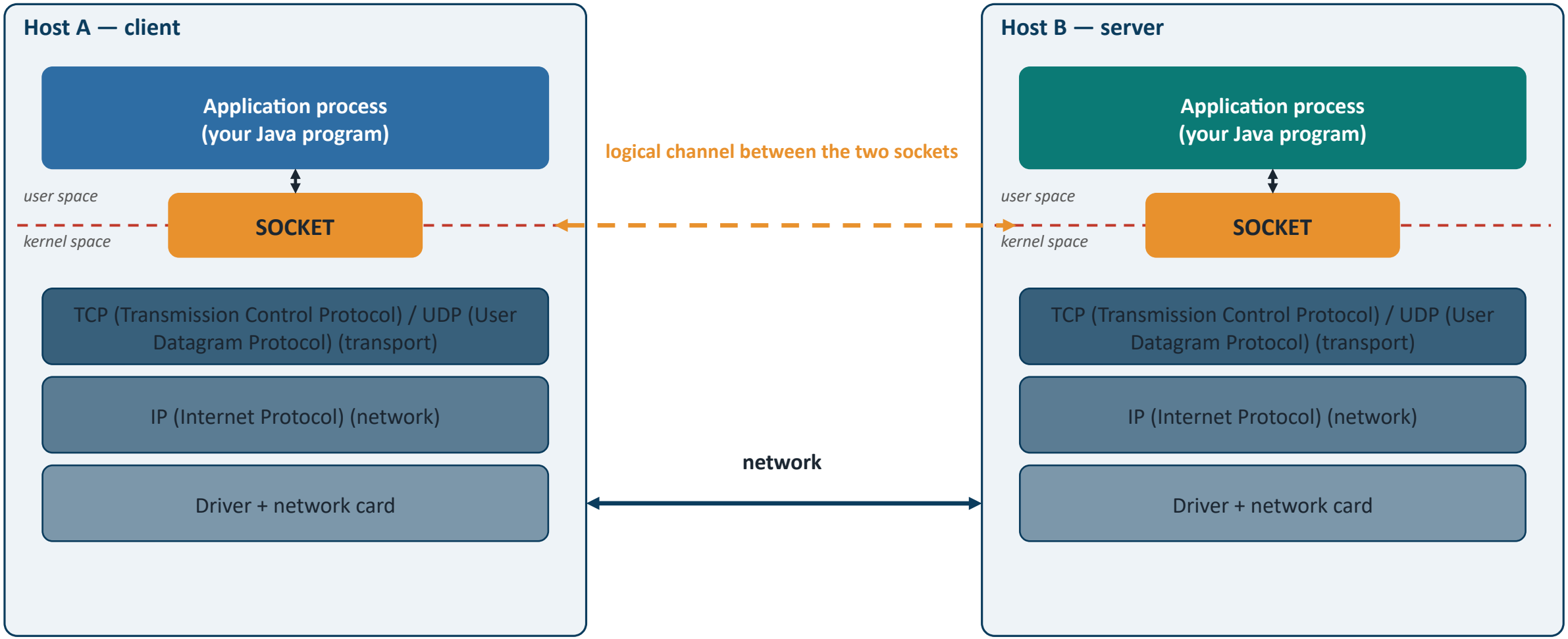
Client

- active: initiates contact
- runs when needed
- OS (Operating System) picks its port

Peer-to-peer (contrast)

- every peer is client AND server
- same socket API (Application Programming Interface) underneath

Where the socket sits: the door between application and kernel



Everything above the socket is the programmer's responsibility; everything below it is done by the OS (Operating System).

2

Sockets: definitions

Endpoint, address, handle, connection

What is a socket? One endpoint, three faces

SOCKET
endpoint of a two-way channel

For the application → a HANDLE

- Linux/Unix: file descriptor (int)
- Windows: SOCKET handle
- Java: a Socket / DatagramSocket object

For the network → a SOCKET ADDRESS

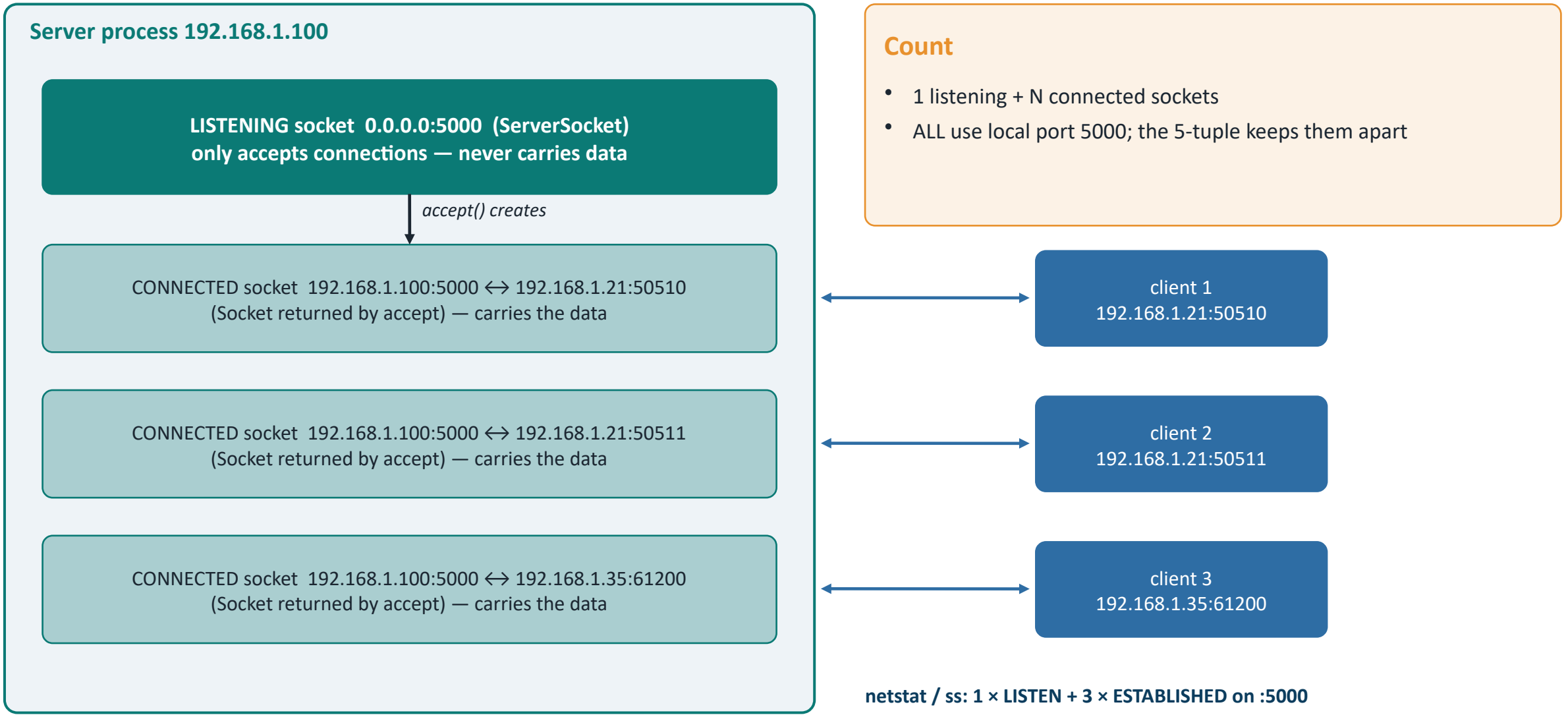
- (IP (Internet Protocol) address, port)
- e.g. 192.168.1.10:5000
- Java: InetSocketAddress

For the OS (Operating System) → a PROTOCOL + state

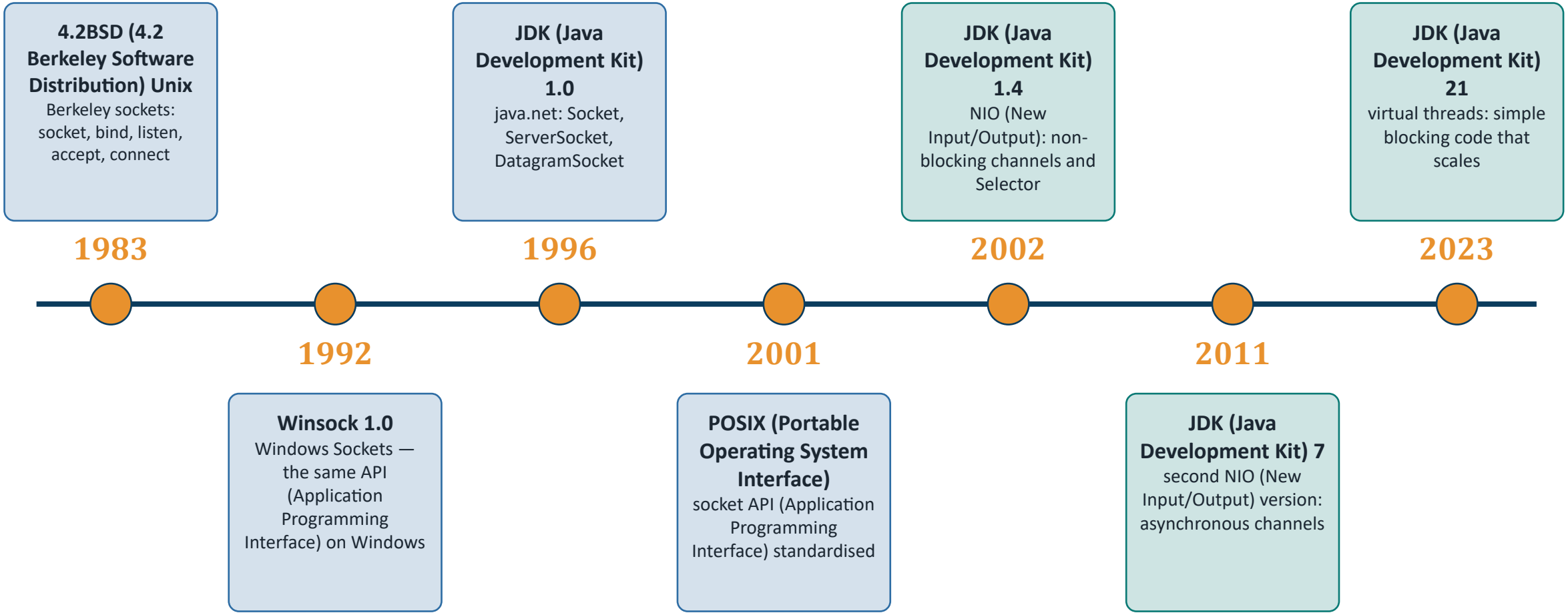
- TCP (Transmission Control Protocol) or UDP (User Datagram Protocol)
- buffers, timers, state (LISTEN, ESTABLISHED…)
- lives in the kernel

Connection (TCP (Transmission Control Protocol)) = pair of sockets = 5-tuple (protocol, local IP (Internet Protocol), local port, remote IP (Internet Protocol), remote port) — the link with Week 6.

Listening socket vs connected sockets (TCP (Transmission Control Protocol) server)



A short history of the socket API (Application Programming Interface)



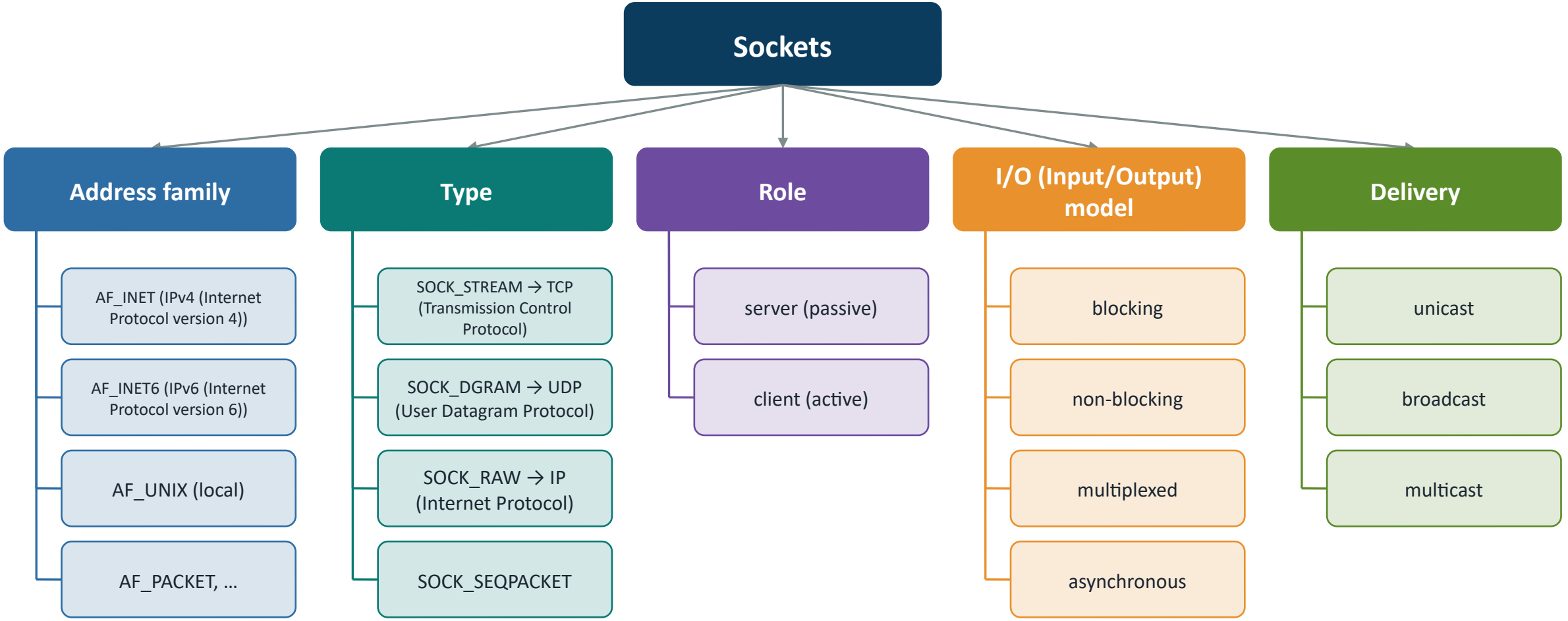
Same primitives for 40 years: what you learn with Java maps directly to C, Python, C#, Go, Rust…

3

Classification of sockets

Five independent criteria

Classification tree: five criteria



This course: Internet sockets (AF_INET/AF_INET6), stream and datagram types, blocking I/O (Input/Output), unicast.

By type: stream vs datagram vs raw sockets

SOCK_STREAM (TCP (Transmission Control Protocol))

H E L L O W O R L D …

a continuous pipe of bytes

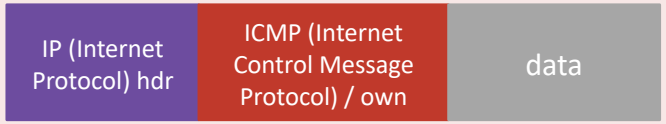
- connection-oriented
- reliable, ordered
- no message boundaries
- Java: Socket, ServerSocket
- web, mail, SSH (Secure Shell), files

SOCK_DGRAM (UDP (User Datagram Protocol))



- connectionless
- independent messages
- boundaries preserved
- may be lost / reordered
- Java: DatagramSocket, DatagramPacket
- DNS (Domain Name System), DHCP (Dynamic Host Configuration Protocol), streaming

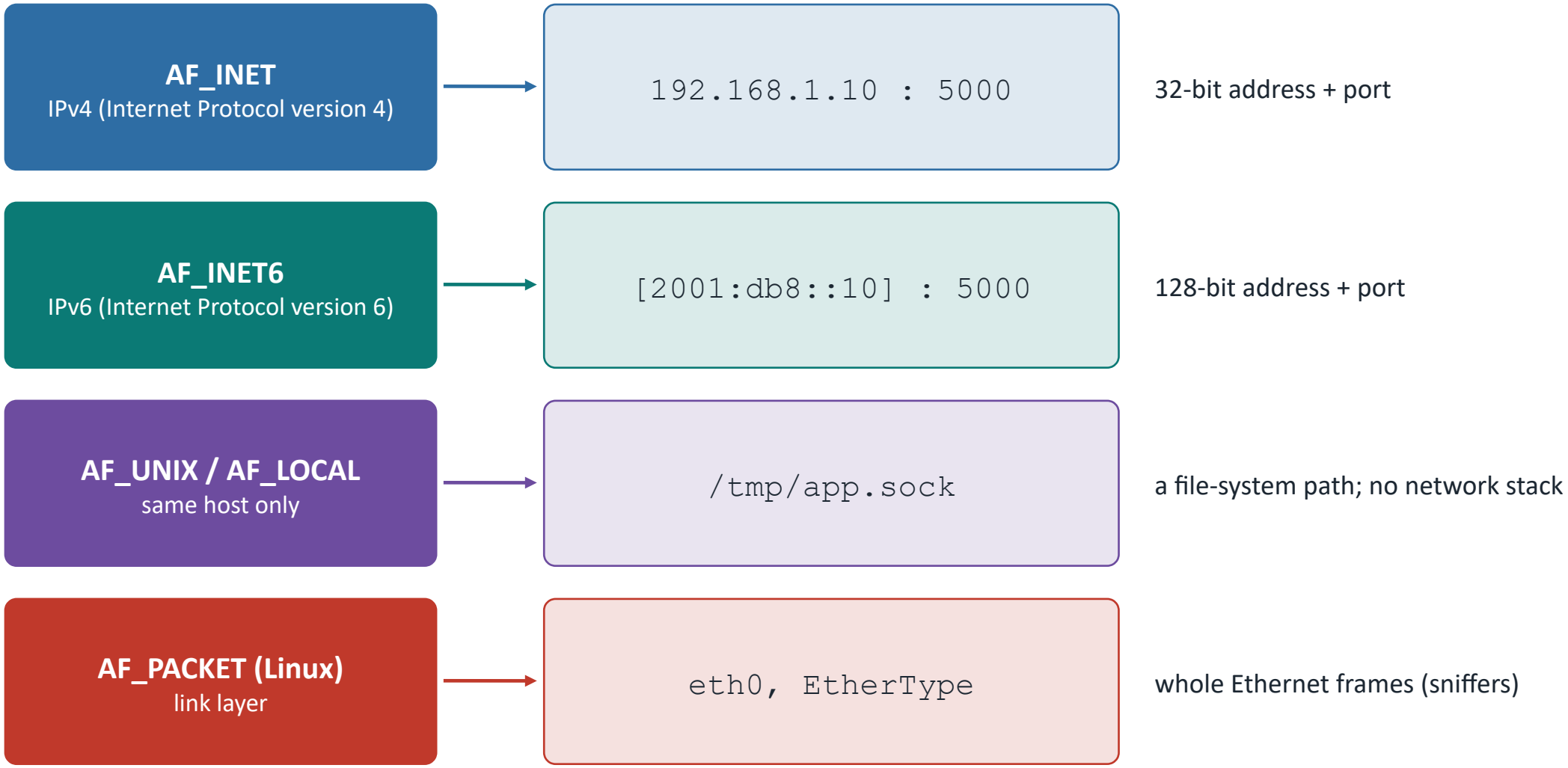
SOCK_RAW (+ SEQPACKET)



the program builds headers itself

- bypasses TCP (Transmission Control Protocol)/UDP (User Datagram Protocol)
- needs admin / root rights
- ping, traceroute, packet tools
- not in java.net
- **SEQPACKET: reliable messages (SCTP (Stream Control Transmission Protocol))**

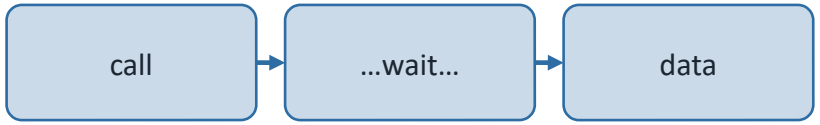
By address family (communication domain)



Java: InetAddress hides IPv4 (Internet Protocol version 4)/IPv6 (Internet Protocol version 6); UnixDomainSocketAddress (Java 16+) for AF_UNIX — also on Windows 10 1803+.

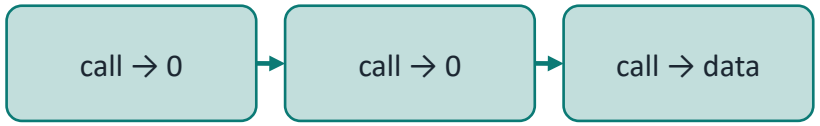
By I/O (Input/Output) model: what happens while the program waits

Blocking (java.net default)



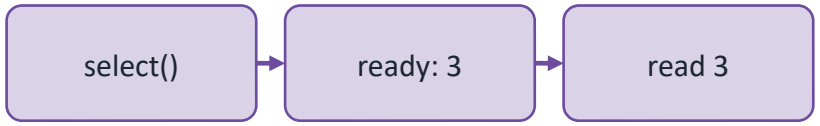
read() suspends the thread until data arrives
simple code; one thread per connection

Non-blocking



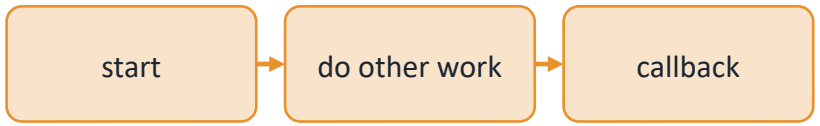
read() returns immediately: data or “nothing yet”
program must poll — wastes CPU (Central Processing Unit) if used alone

Multiplexed



one thread asks: which of my sockets are ready?
select/poll/epoll, Windows IOCP (Input/Output Completion Ports), Java NIO (New Input/Output) Selector

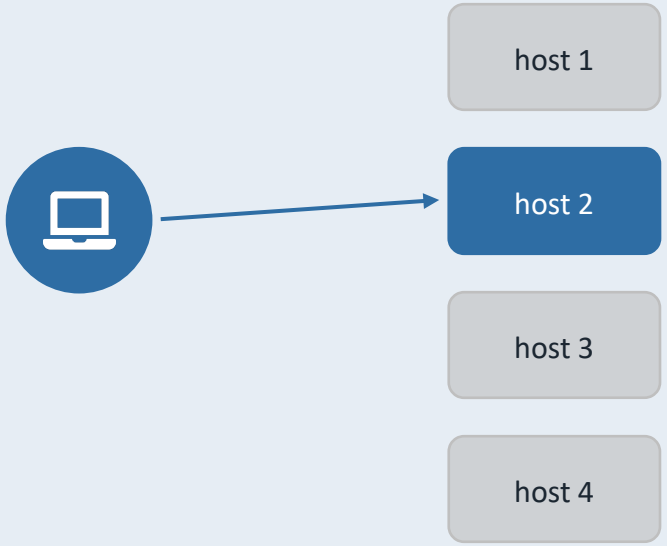
Asynchronous



start the read; the OS (Operating System) calls you back when done
Java AsynchronousSocketChannel, callbacks/futures

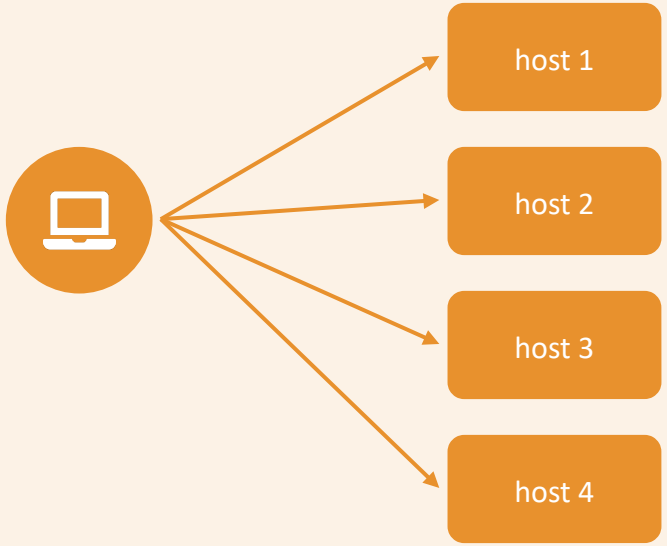
By delivery mode: unicast, broadcast, multicast

Unicast



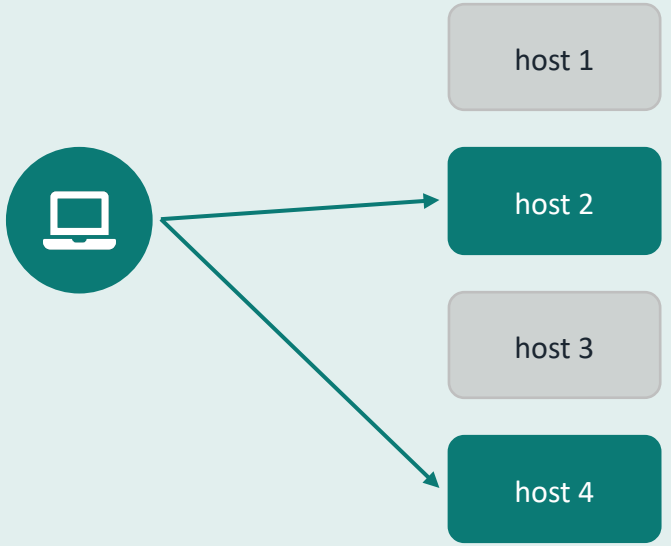
one sender → one receiver
TCP (Transmission Control Protocol) and UDP (User Datagram Protocol)

Broadcast



one sender → all hosts of the subnet
UDP (User Datagram Protocol) only — DatagramSocket.setBroadcast(true), e.g. 255.255.255.255 (DHCP (Dynamic Host Configuration Protocol))

Multicast



one sender → hosts that joined a group
UDP (User Datagram Protocol) only — MulticastSocket.joinGroup(224.x.x.x – 239.x.x.x)

4

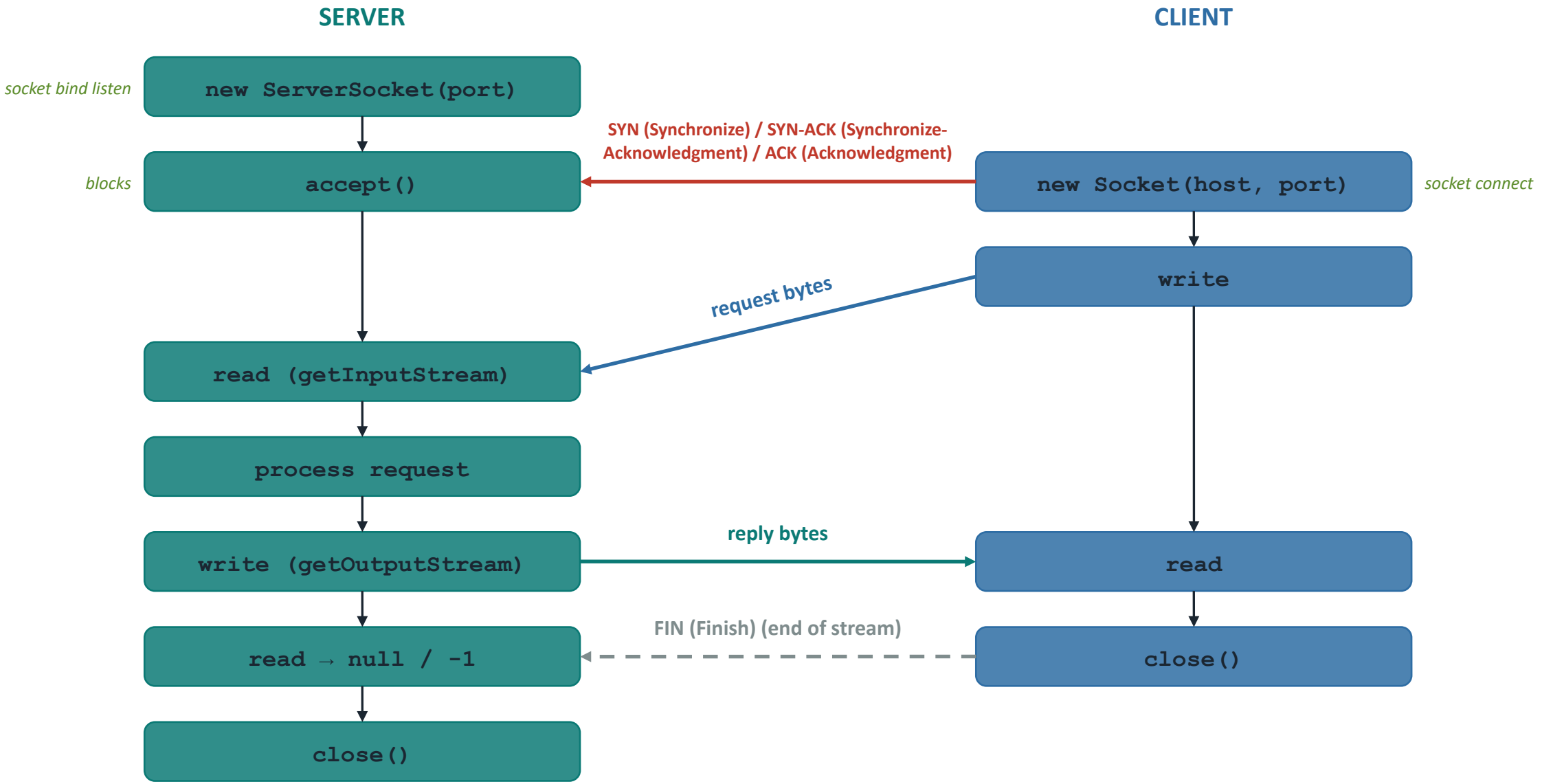
The socket API (Application Programming Interface) and call sequences

Berkeley primitives and what each one sends on the wire

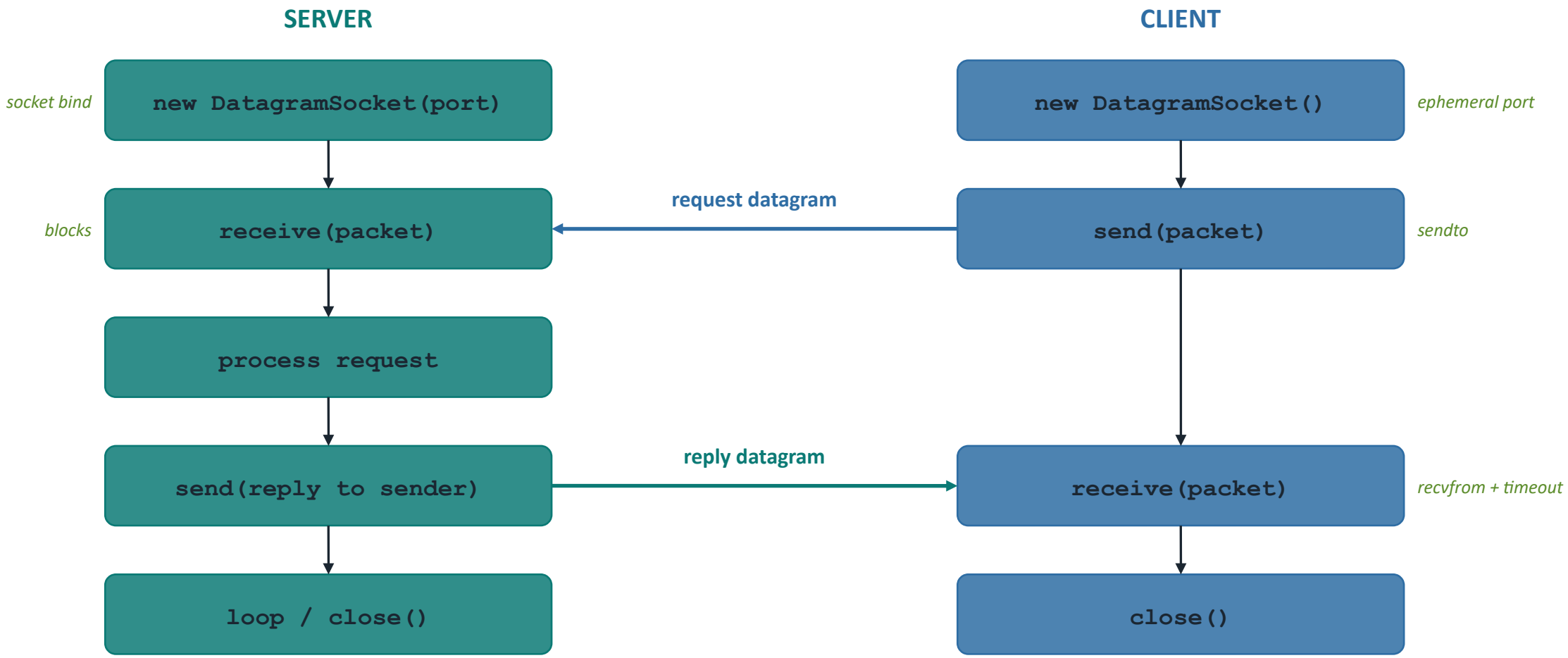
The Berkeley socket primitives

Primitive	Purpose	On the wire	Used by
`socket()`	create a socket (family, type)	—	both
`bind()`	attach local IP (Internet Protocol) + port	—	servers
`listen()`	passive mode + backlog queue	port in LISTEN	TCP (Transmission Control Protocol) server
`accept()`	take a finished connection → new socket	(handshake done by kernel)	TCP (Transmission Control Protocol) server
`connect()`	open connection to remote address	SYN (Synchronize) → SYN-ACK (Synchronize-Acknowledgment) → ACK (Acknowledgment)	TCP (Transmission Control Protocol) client
`send()` / `recv()`	bytes on a connected socket	data segments + ACKs (Acknowledgments)	TCP (Transmission Control Protocol)
`sendto()` / `recvfrom()`	one datagram to / from an address	one UDP (User Datagram Protocol) datagram	UDP (User Datagram Protocol)
`shutdown()`	close one direction	FIN (Finish)	TCP (Transmission Control Protocol)
`close()`	release the socket	FIN (Finish) (or RST (Reset))	both

TCP (Transmission Control Protocol): the sequence of calls (Java names, POSIX (Portable Operating System Interface) calls in green)

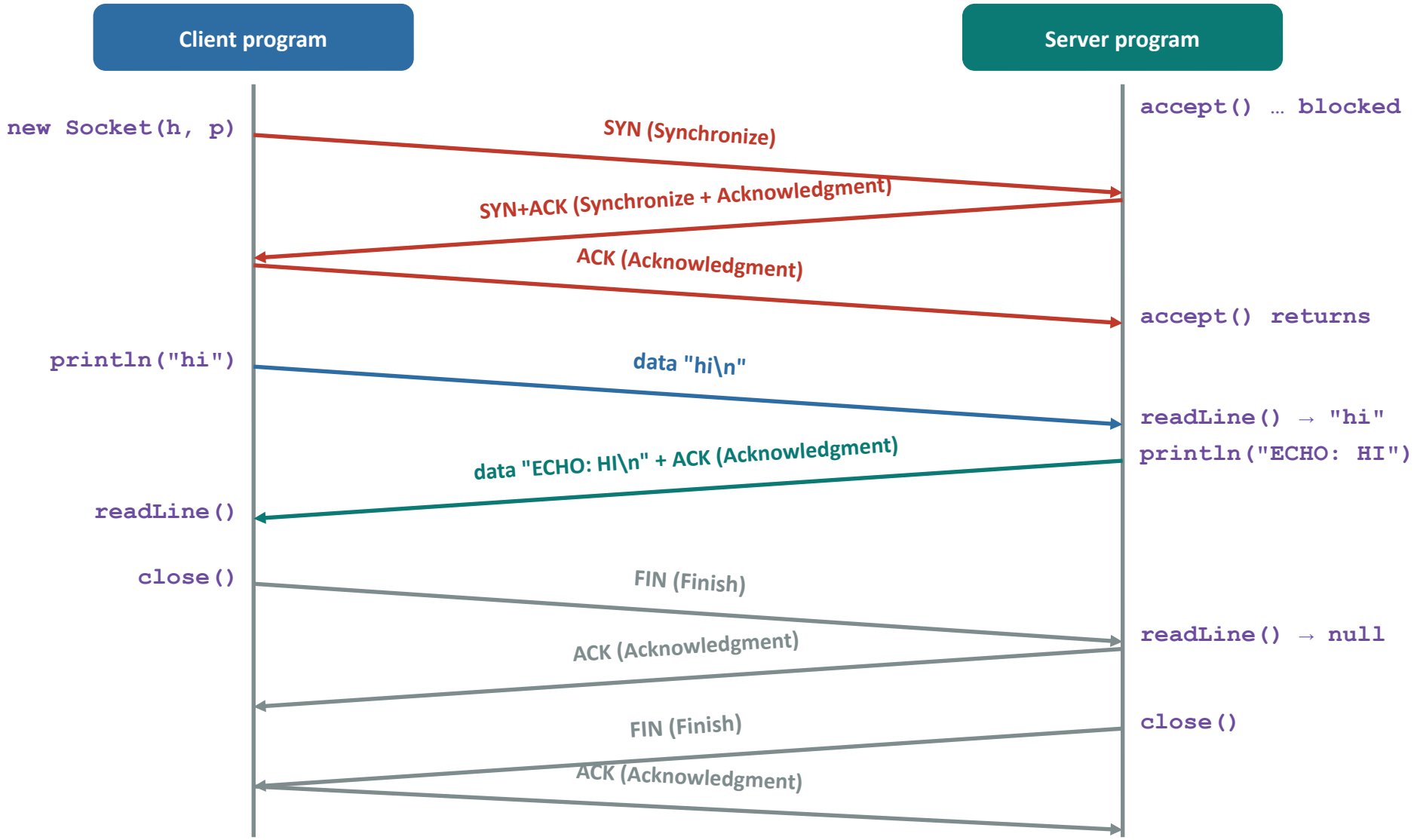


UDP (User Datagram Protocol): the sequence of calls — no connection, no accept

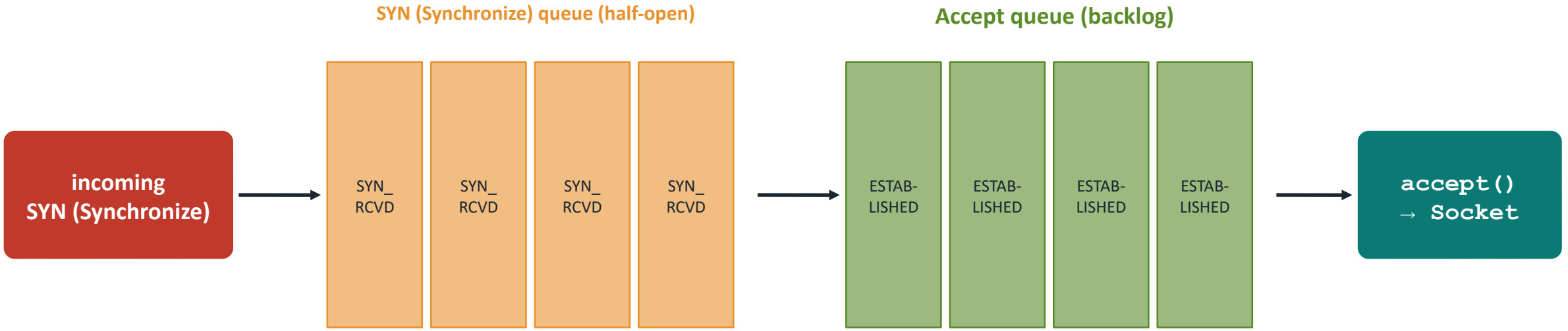


One server socket for all clients · the reply address is taken from the received packet

From calls to packets: one TCP (Transmission Control Protocol) echo session



Inside listen() and accept(): the two kernel queues



the final ACK (Acknowledgment) of the handshake moves a connection from the SYN (Synchronize) queue to the accept queue

What it means

- the handshake is completed by the KERNEL, even before accept() is called
- accept() just takes the next ready connection
- new ServerSocket(port, backlog) sets the accept-queue length (Java default 50)

When the queue is full

- new SYNs (Synchronize segments) are dropped (Linux) or refused
- clients see slow or failed connects
- → call accept() quickly; hand clients to threads

5

Network programming in Java

The java.net classes, the API (Application Programming Interface) mapping and the exceptions

Map of java.net for socket programming

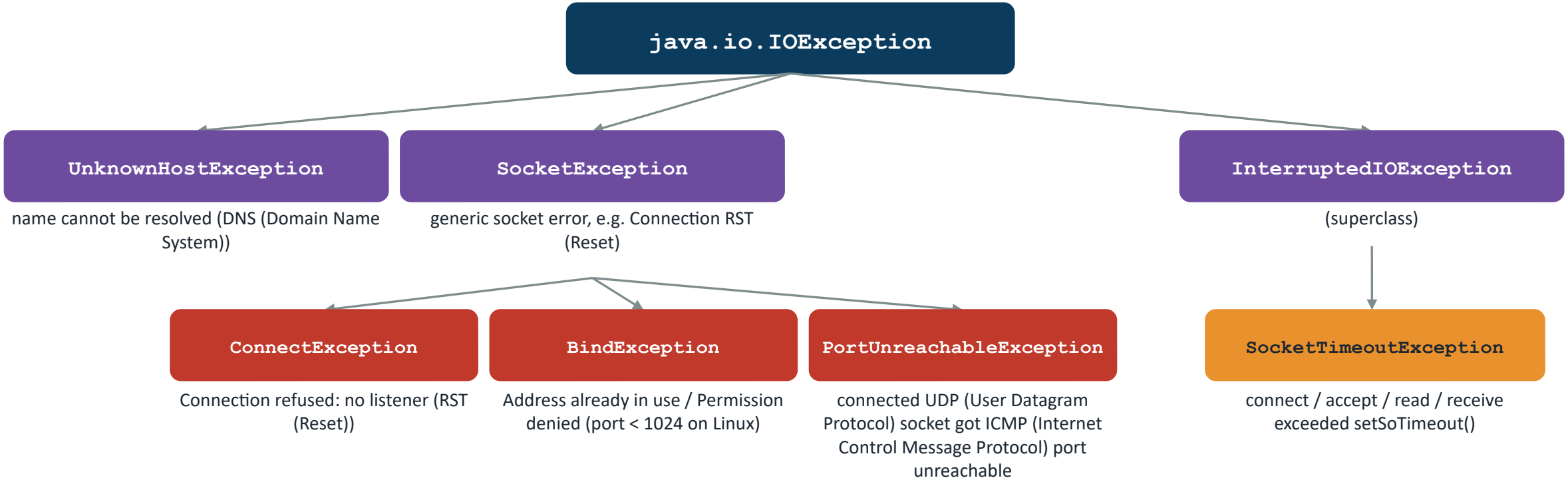
Addresses	TCP (Transmission Control Protocol) (stream)	UDP (User Datagram Protocol) (datagram)	Beyond this course
`InetAddress` IP (Internet Protocol) address + DNS (Domain Name System) lookup	**`ServerSocket`** listening socket: bind + listen + accept()	**`DatagramSocket`** send() / receive() datagrams	**`java.nio.channels`** SocketChannel, Selector (non-blocking)
`Inet4Address / Inet6Address` concrete subclasses	**`Socket`** connected socket: streams in/out	**`DatagramPacket`** one datagram: data + length + address	**`AsynchronousSocketChannel`** callbacks / futures
`InetSocketAddress` IP (Internet Protocol) address + port		**`MulticastSocket`** joinGroup() / leaveGroup()	**`java.net.http.HttpClient`** a whole protocol (HTTP (Hypertext Transfer Protocol))

```
import java.net.*;   import java.io.*;   — all classes used in the lab are here.
```

From the C socket API (Application Programming Interface) to Java

C / POSIX (Portable Operating System Interface) / Winsock		Java (java.net)
`socket() + bind() + listen()`	→	`new ServerSocket(port, backlog)`
`accept()`	→	`serverSocket.accept() → Socket`
`socket() + connect()`	→	`new Socket(host, port)`
`send() / recv()`	→	`getOutputStream().write() / getInputStream().read()`
`socket() + bind() (UDP (User Datagram Protocol))`	→	`new DatagramSocket(port)`
`sendto()`	→	`datagramSocket.send(packet)`
`recvfrom()`	→	`datagramSocket.receive(packet)`
`setsockopt(SO_RCVTIMEO)`	→	`setSoTimeout(ms)`
`shutdown(SHUT_WR) / close()`	→	`shutdownOutput() / close()`
`gethostbyname() / getaddrinfo()`	→	`InetAddress.getByName(name)`

Networking exceptions: hierarchy and causes



Good practice: catch the specific exception, tell the user what happened (wrong host? server down? port busy? timeout?), and always close sockets in try-with-resources so that a failure never leaves a half-open connection behind.

InetAddress: names, addresses and DNS (Domain Name System)

```
import java.net.*;

public class AddressInfo {
  public static void main(String[] a) throws Exception {
    String name = "www.example.com";
    for (InetAddress x : InetAddress.getAllByName(name)) // DNS (Domain
Name System)
      System.out.println(name + " -> " + x.getHostAddress());
    System.out.println(InetAddress.getLocalHost());
    System.out.println(InetAddress.getLoopbackAddress());
  }
}
```

possible output

```
www.example.com -> 93.184.215.14
www.example.com -> 2606:2800:21f:cb07:6820:80da:af6b:8b2c
lab-pc-07/192.168.10.21
localhost/127.0.0.1
```

1. **getByName / getAllByName**
   resolves a host name (A + AAAA records); throws UnknownHostException

2. **getHostAddress()**
   the numeric form as text

3. **getLocalHost()**
   this machine's name and address

4. **getLoopbackAddress()**
   127.0.0.1 (or ::1) — tests on one PC (Personal Computer)

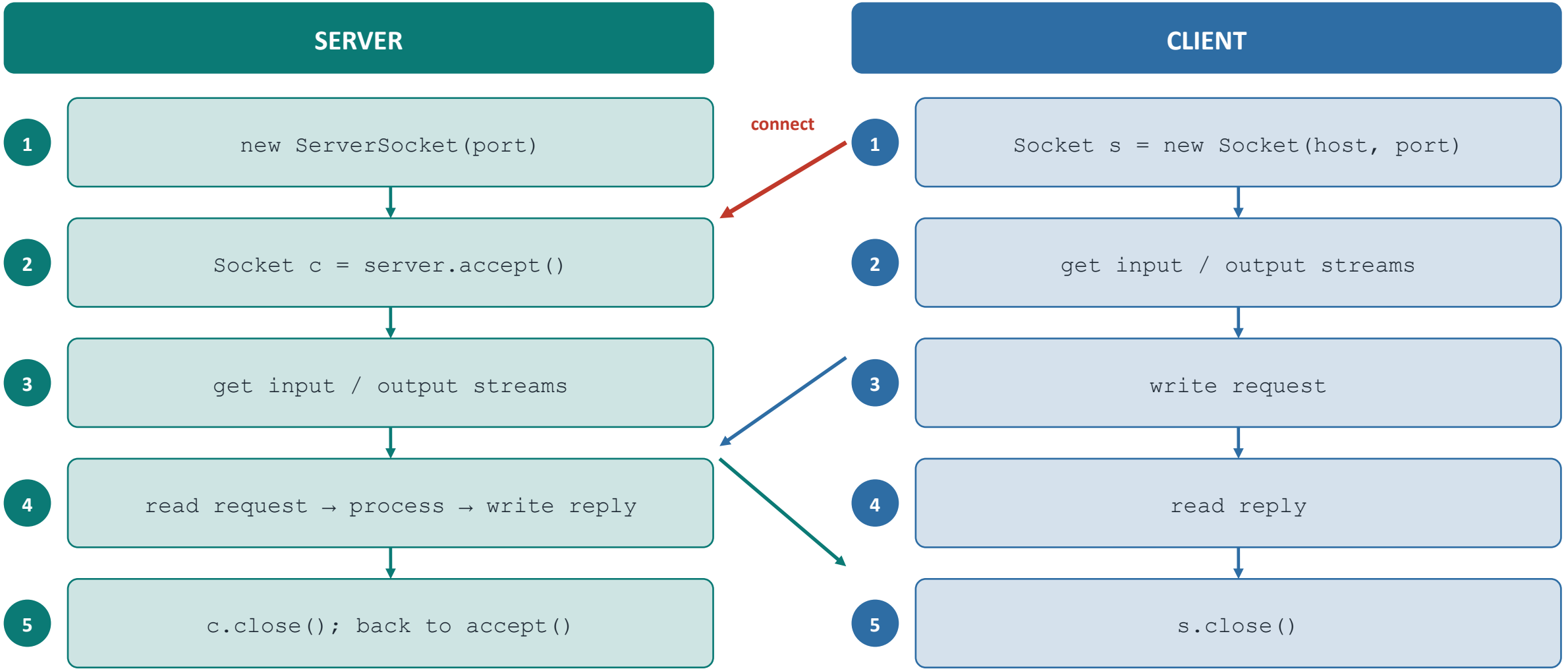
5. **IPv4 (Internet Protocol version 4) or IPv6 (Internet Protocol version 6)**
   the same code works for both: Inet4Address / Inet6Address

6

TCP (Transmission Control Protocol) client-server in Java

ServerSocket, Socket, streams, concurrency and framing

TCP (Transmission Control Protocol) in Java: the steps on both sides



TcpEchoServer.java — annotated

```
public class TcpEchoServer {
  public static void main(String[] args) throws IOException {
    int port = args.length > 0 ? Integer.parseInt(args[0]) : 5000;
    try (ServerSocket server = new ServerSocket(port)) {   // (1)
      while (true) {
        Socket client = server.accept();                   // (2)
        new Thread(() -> handle(client)).start();          // (3)
      }
    }
  }
  static void handle(Socket s) {
    try (s;
         BufferedReader in = new BufferedReader(
           new InputStreamReader(s.getInputStream(), "UTF-8"));   // (4)
         PrintWriter out = new PrintWriter(
           new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true)) {
      String line;
      while ((line = in.readLine()) != null) {             // (5)
        if (line.equalsIgnoreCase("BYE")) { out.println("Goodbye"); break; }
        out.println("ECHO: " + line.toUpperCase());        // (6)
      }
    } catch (IOException e) { System.out.println("Error: " + e.getMessage()); }
  }                                                        // (7) auto-close
}
```

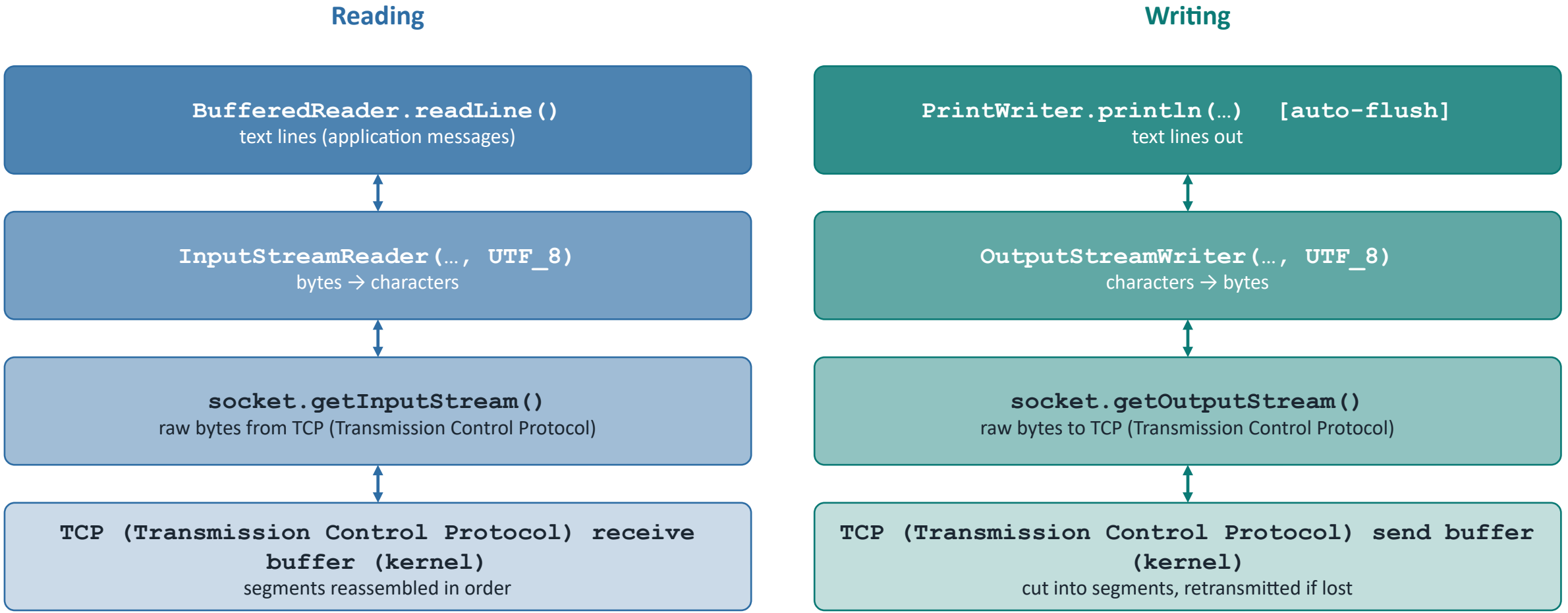
1. **ServerSocket(port)**
   socket + bind + listen; port now LISTEN
2. **accept()**
   blocks; returns a NEW connected Socket
3. **one thread per client**
   the loop goes back to accept() at once
4. **streams + UTF-8 (8-bit Unicode Transformation Format)**
   bytes ↔ text, same on Windows/Linux
5. **readLine()**
   a message = one line; null = peer sent FIN (Finish)
6. **println + auto-flush**
   reply is pushed to TCP (Transmission Control Protocol) immediately
7. **try-with-resources**
   closes streams and socket → FIN (Finish)

TcpEchoClient.java — annotated

```
public class TcpEchoClient {
  public static void main(String[] args) throws IOException {
    String host = args.length > 0 ? args[0] : "localhost";
    int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;
    try (Socket s = new Socket(host, port);                    // (1)
         BufferedReader in = new BufferedReader(
           new InputStreamReader(s.getInputStream(), "UTF-8"));
         PrintWriter out = new PrintWriter(
           new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true);
         BufferedReader kb = new BufferedReader(
           new InputStreamReader(System.in))) {                // (2)
      System.out.println("Local port: " + s.getLocalPort());  // (3)
      String line;
      while ((line = kb.readLine()) != null) {
        out.println(line);                                     // (4)
        String reply = in.readLine();                          // (5)
        if (reply == null) break;
        System.out.println(reply);
        if (line.equalsIgnoreCase("BYE")) break;               // (6)
      }
    }                                                          // (7)
  }
}
```

1. **new Socket(host, port)**
   DNS (Domain Name System) + ephemeral port + three-way handshake
2. **keyboard reader**
   lines typed by the user
3. **getLocalPort()**
   the ephemeral port chosen by the OS (Operating System)
4. **println(line)**
   request = one line, flushed at once
5. **readLine()**
   waits for the server's reply line
6. **BYE**
   application-level end of session
7. **auto-close**
   close() sends FIN (Finish)

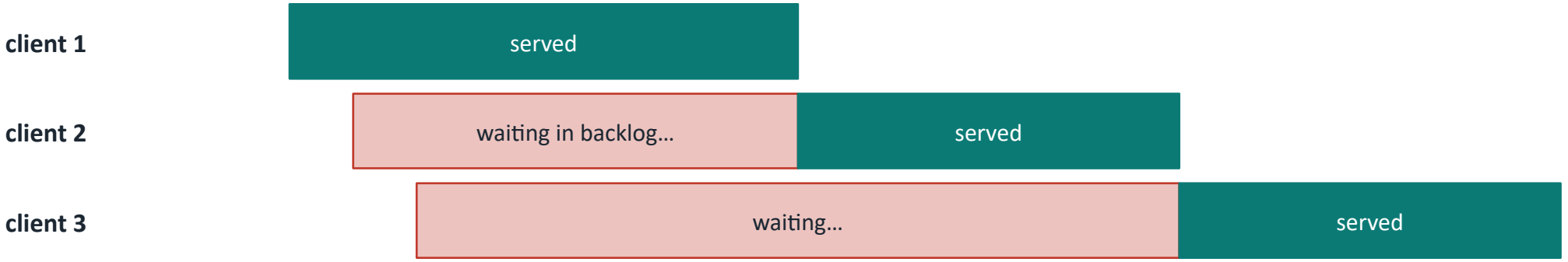
Stream layers on a TCP (Transmission Control Protocol) socket



Alternatives: DataInputStream / DataOutputStream for binary data (readInt, writeInt, readFully).

Iterative vs concurrent server: who waits?

Iterative server: one client at a time



Concurrent server: a thread per client



Same work, but the concurrent server finishes all three clients by t = 4 instead of t = 10.

Concurrency models for servers

Thread per client

```
accept → new Thread(handle).start()
```

simple; but thousands of clients = thousands of OS (Operating System) threads

Thread pool

```
accept → pool.submit(() -> handle(c))
Executors.newFixedThreadPool(50)
```

bounded resources; pool size limits concurrency

Virtual threads (Java 21+)

```
Executors.newVirtualThreadPerTaskExecutor()
```

blocking code style that scales to very many clients

Event-driven (NIO (New Input/Output))

```
one thread + Selector over non-blocking channels
```

highest scalability; more complex code

Message framing: TCP (Transmission Control Protocol) has no message boundaries

Sender: write("HELLO") then write("WORLD") → receiver's read(buf) may return…

HELLOWORLD	HELLO | WORLD	HEL | LOWORLD
both in one read	*as written (lucky)*	*split anywhere*

The application must define where a message ends — three solutions:

Delimiter	`HELLO\n WORLD\n`	readLine(); used by HTTP (Hypertext Transfer Protocol) headers, SMTP (Simple Mail Transfer Protocol), our echo protocol
Length prefix	`[00 00 00 05] HELLO [00 00 00 05] WORLD`	writeInt(len) + bytes; readInt() + readFully(); binary protocols
Fixed size	`every message exactly N bytes`	readFully(buf, 0, N); simple but rigid

Length-prefix framing in Java




```
// sender
DataOutputStream out = new DataOutputStream(
    new BufferedOutputStream(sock.getOutputStream()));
byte[] m = text.getBytes(StandardCharsets.UTF_8);
out.writeInt(m.length);   out.write(m);   out.flush();
```

```
// receiver
DataInputStream in = new DataInputStream(
    new BufferedInputStream(sock.getInputStream()));
int len = in.readInt();
byte[] b = new byte[len];  in.readFully(b);
```

Why readFully()?

- read(buf) may return FEWER bytes than requested (whatever has arrived); readFully loops until exactly len bytes are in the buffer
- readInt() itself also waits for all 4 bytes → the message boundary is always found

Useful TCP (Transmission Control Protocol) socket options

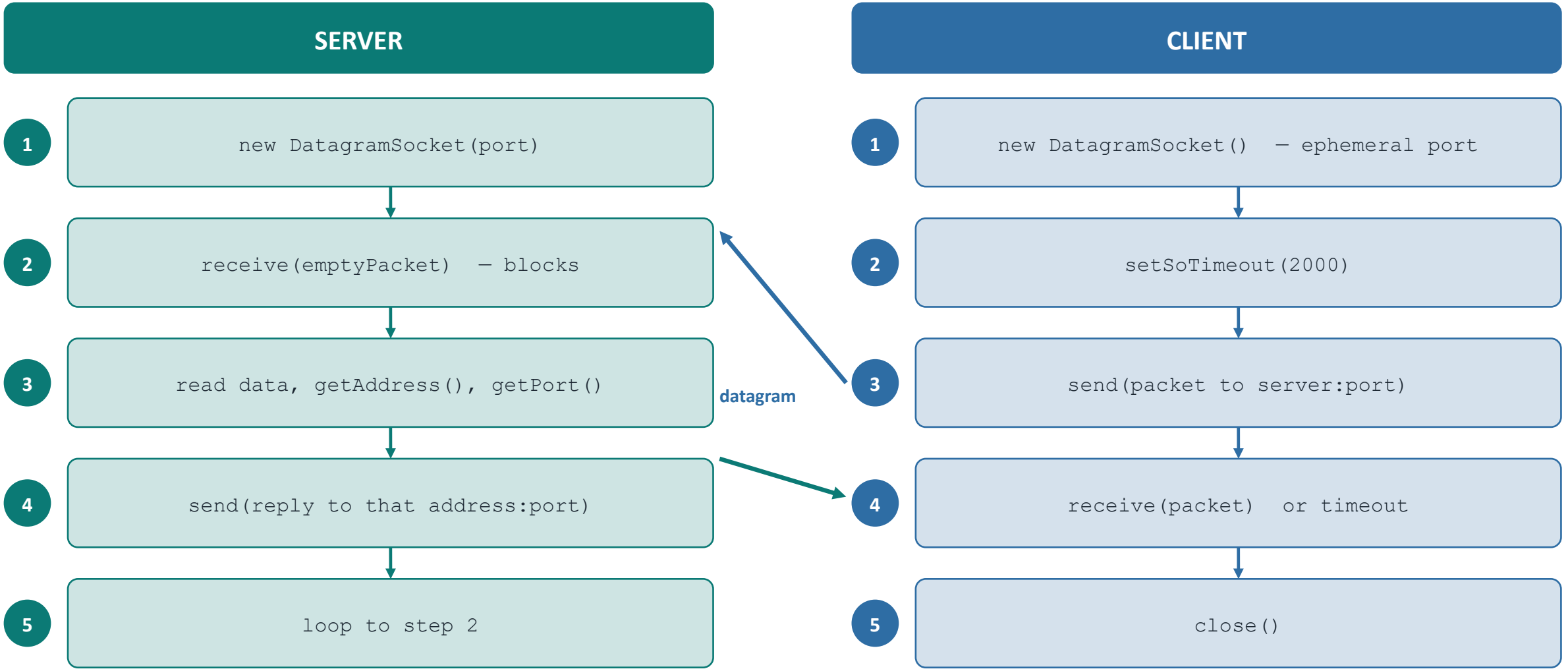
`setSoTimeout(ms)`	max blocking time of read / accept → *SocketTimeoutException*
`connect(addr, ms)`	limit the handshake time *new Socket() then connect()*
`setTcpNoDelay(true)`	disable Nagle: small writes go out at once *interactive apps, games*
`setKeepAlive(true)`	probes on idle connections *detects dead peers*
`setReuseAddress(true)`	rebind a port with TIME_WAIT leftovers *restart servers quickly*
`set…BufferSize(n)`	kernel buffer size *affects advertised rwnd*
`shutdownOutput()`	half-close: send FIN (Finish), keep reading *"I'm done, give me the answer"*
`ServerSocket(p, backlog, addr)`	queue length; bind to one interface *127.0.0.1 = local only*

7

UDP (User Datagram Protocol) client-server in Java

DatagramSocket, DatagramPacket and reliability by hand

UDP (User Datagram Protocol) in Java: the steps on both sides



No ServerSocket, no accept(), no streams — just packets.

UdpEchoServer.java — annotated

```
public class UdpEchoServer {
  public static void main(String[] args) throws Exception {
    int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
    try (DatagramSocket socket = new DatagramSocket(port)) {    // (1)
      byte[] buf = new byte[1024];
      while (true) {
        DatagramPacket req = new DatagramPacket(buf, buf.length); // (2)
        socket.receive(req);                                      // (3)
        String msg = new String(req.getData(), 0, req.getLength(),
                                StandardCharsets.UTF_8);          // (4)
        byte[] ans = ("ECHO: " + msg.toUpperCase())
                       .getBytes(StandardCharsets.UTF_8);
        socket.send(new DatagramPacket(ans, ans.length,
                    req.getAddress(), req.getPort()));            // (5)
      }
    }
  }
}
```

1. **DatagramSocket(port)**
   socket + bind on a known port

2. **empty packet**
   a 1024-byte buffer to receive into

3. **receive()**
   blocks; fills buffer, length and sender address

4. **getLength()!**
   only the bytes of THIS datagram

5. **reply to sender**
   address and port taken from the request

One socket serves every client — no per-client state, no threads needed for short requests

UdpEchoClient.java — annotated

```
public class UdpEchoClient {
  public static void main(String[] args) throws Exception {
    InetAddress server = InetAddress.getByName(
        args.length > 0 ? args[0] : "localhost");              // (1)
    int port = args.length > 1 ? Integer.parseInt(args[1]) : 6000;
    try (DatagramSocket socket = new DatagramSocket();          // (2)
         BufferedReader kb = new BufferedReader(
           new InputStreamReader(System.in))) {
      socket.setSoTimeout(2000);                                // (3)
      String line;
      while ((line = kb.readLine()) != null && !line.equalsIgnoreCase("BYE")) {
        byte[] data = line.getBytes(StandardCharsets.UTF_8);
        socket.send(new DatagramPacket(data, data.length,
                                       server, port));          // (4)
        DatagramPacket resp = new DatagramPacket(new byte[1024], 1024);
        try {
          socket.receive(resp);                                 // (5)
          System.out.println(new String(resp.getData(), 0,
                     resp.getLength(), StandardCharsets.UTF_8));
        } catch (SocketTimeoutException e) {                    // (6)
          System.out.println("No reply within 2 s");
        }
      }
    }
  }
}
```

1. **resolve the server**
   InetAddress for the destination

2. **DatagramSocket()**
   no port given → ephemeral port

3. **setSoTimeout**
   UDP (User Datagram Protocol) gives no guarantee: never wait forever

4. **send(packet)**
   data + destination address + port

5. **receive(resp)**
   the reply datagram (fresh packet)

6. **timeout**
   request or reply lost, or server down

Anatomy of a DatagramPacket

A packet object = buffer + offset + length + remote address + remote port



buffer of 1024 bytes (only 16 drawn)

getLength() = 5

old bytes from an earlier, longer message

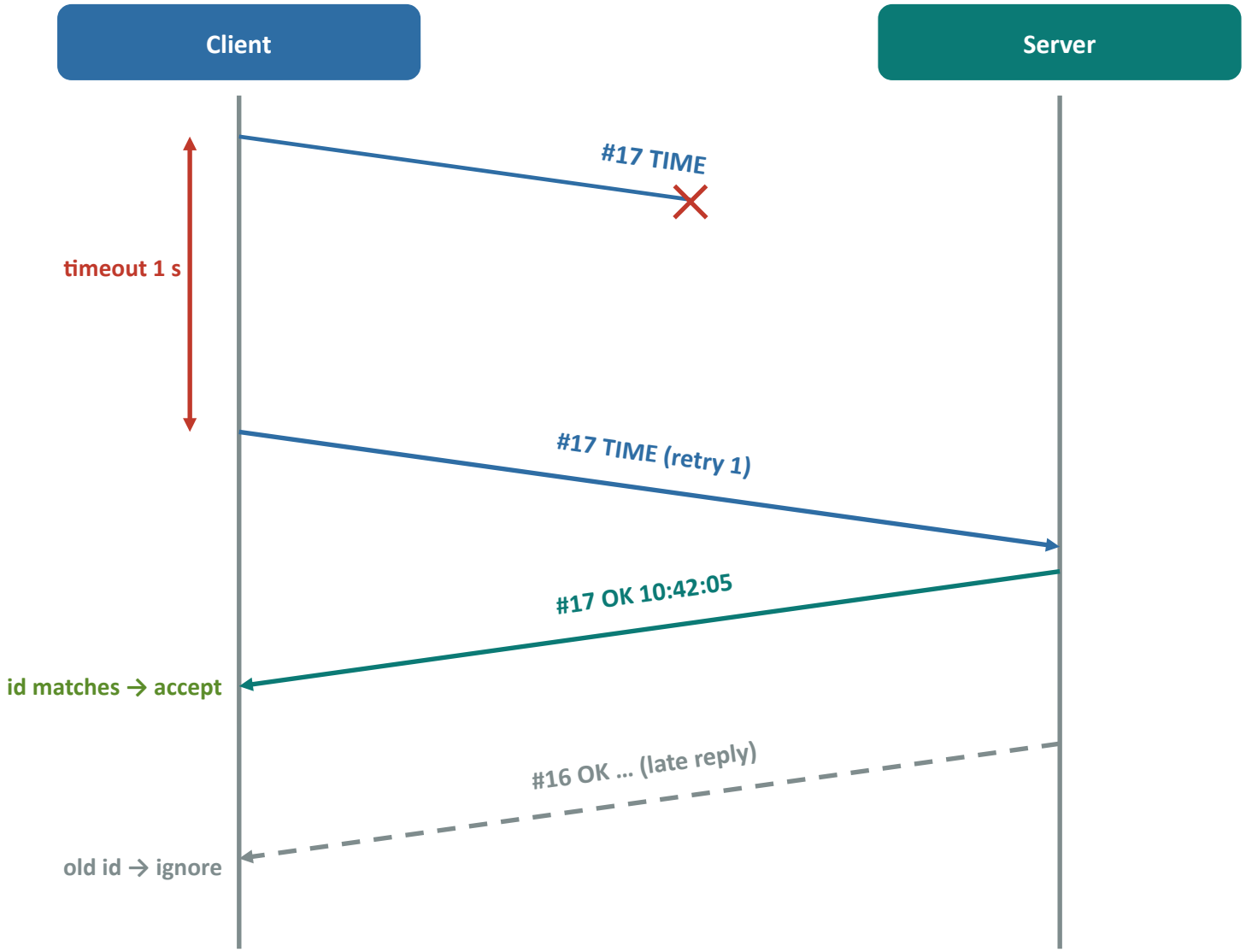
Receiving

- new DatagramPacket(buf, buf.length)
- after receive(): getData() = whole buffer, getLength() = size of this datagram
- getAddress() / getPort() = the SENDER
- receive() shrinks the packet's length → reuse needs setLength(buf.length)
- datagram larger than the buffer → extra bytes silently lost

Sending

- new DatagramPacket(data, data.length, address, port)
- address + port = the DESTINATION
- one send() = exactly one UDP (User Datagram Protocol) datagram
- correct decoding:
- new String(p.getData(), 0, p.getLength(), UTF_8)

Making UDP (User Datagram Protocol) reliable enough: timeout, retry, request id



Recipe (assignment A2)

- put a request id in every message
- wait with setSoTimeout()
- on timeout: resend, at most N times (e.g. 3)
- ignore replies with another id
- prefer idempotent requests (TIME, ADD 3 4): executing twice is harmless
- this is a mini stop-and-wait ARQ (Automatic Repeat reQuest) — Week 6!

How big can a datagram be?

IPv4 (Internet Protocol version 4) packet max: 65,535 bytes

UDP (User Datagram Protocol) payload max: 65,535 − 20 (IP (Internet Protocol)) − 8 (UDP (User Datagram Protocol)) = 65,507 bytes

1500

Ethernet MTU (Maximum Transmission Unit): 1500 bytes

1472

safe UDP (User Datagram Protocol) payload on Ethernet: 1500 − 20 − 8 = 1472 bytes

Bigger than the MTU (Maximum Transmission Unit)?

- IP (Internet Protocol) fragments the datagram
- losing ONE fragment loses the WHOLE datagram

Rule of thumb

- keep datagrams ≤ ~1200–1400 bytes
- need more? split it yourself or use TCP (Transmission Control Protocol)

8

TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol) programming in practice

Comparison, error behaviour, troubleshooting, testing

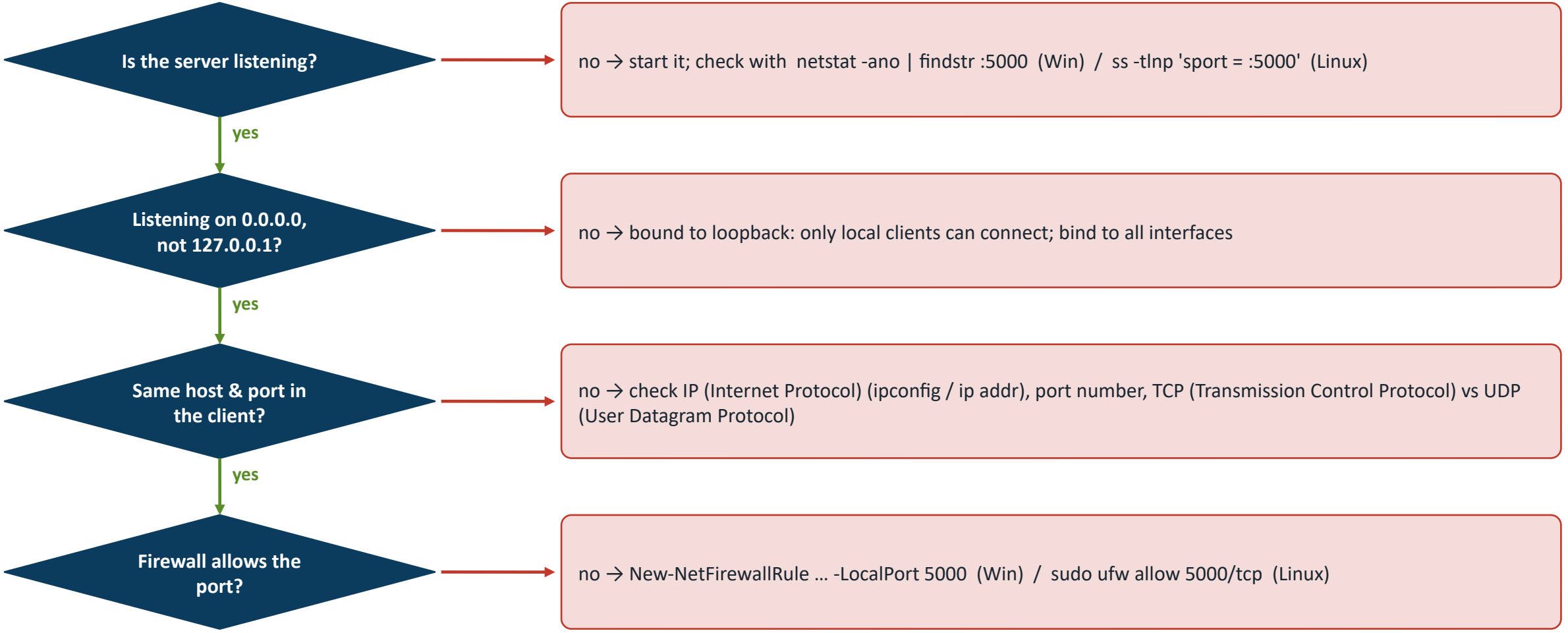
TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol) programming — side by side

	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Java classes	ServerSocket, Socket	DatagramSocket, DatagramPacket
Connection set-up	yes: new Socket() / accept()	none
Sockets on the server	1 listening + 1 per client	1 for all clients
Data unit	byte stream → needs framing	datagram → boundaries kept
Reliability, order	guaranteed by TCP (Transmission Control Protocol)	application's job
I/O (Input/Output) style	InputStream / OutputStream	send() / receive() packets
End of communication	read() → -1 / readLine() → null	not visible: timeouts or a BYE message
Broadcast / multicast	impossible	possible
Typical server	concurrent (threads)	single loop

What does the program see when things go wrong?

	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Server not running	ConnectException: Connection refused (RST (Reset))	send() OK; receive() times out (connected socket: PortUnreachableException)
Host down / firewall drops	connect() hangs, then times out	receive() times out
Port already in use	BindException: Address already in use	BindException: Address already in use
Port < 1024 on Linux (no root)	BindException: Permission denied	BindException: Permission denied
Server killed mid-session	readLine() → null (FIN (Finish)) or Connection RST (Reset)	receive() times out
Wrong host name	UnknownHostException	UnknownHostException

Troubleshooting flowchart: “my client cannot connect”



all yes → capture with Wireshark / tcpdump: SYN (Synchronize) without answer = filtered; RST (Reset) = closed port; SYN-ACK (Synchronize-Acknowledgment) = network OK

Testing toolkit: Windows and Linux

	Windows (PowerShell)	Linux (bash)
Compile / run	`javac *.java · java TcpEchoServer 5000`	`javac *.java · java TcpEchoServer 5000`
Who listens?	`netstat -ano | findstr :5000` `Get-NetTCPConnection -LocalPort 5000`	`ss -tlnp 'sport = :5000'` `ss -uanp 'sport = :6000'`
Generic client	`ncat localhost 5000 · ncat -u … 6000` `Test-NetConnection localhost -Port 5000`	`nc localhost 5000 · nc -u … 6000` `nc -zv host 5000`
Open the firewall	`New-NetFirewallRule -DisplayName "CN 5000"` `-Direction Inbound -Protocol TCP -LocalPort 5000 -Action Allow`	`sudo ufw allow 5000/tcp` `sudo firewall-cmd --add-port=5000/tcp`
Watch packets	`Wireshark (Npcap loopback adapter)` `filter: tcp.port == 5000`	`sudo tcpdump -i lo -nn -A 'tcp port 5000'`
Threads of the server	`jcmd · jstack <pid>`	`jcmd · jstack <pid> · ps -o nlwp= -p <pid>`

Summary: socket communication on one page

